POLITECNICO
MILANO 1863

AN2DL - First Challenge Report
The Underfitters

Benkirane Ilyas, Lecomte Anatole, Luneau Nathan

295150, 294859, 295387

December 17, 2025

1 Introduction

This report details the approach that our team, *The Underfitters*, followed for the first project of the course **Artificial Neural Networks and Deep Learning**.

The goal of this assignment is to address a *multivariate time series classification problem* using **deep learning models** trained entirely from scratch. The objective is to design a model that can learn meaningful patterns and accurately predict the pain level of subjects in a dataset composed of Pirates.
The project forbids pretrained architectures or weights, and the evaluation metric is the *F1-score*, requiring balanced performance across classes.

Our approach was inspired by the lectures and practical exercises, particularly **Lab 4: Time-Series Classification**, which formed the basis of our solution. We followed the course guidance and recommendations to shape our model development.

2 Problem Analysis

Dataset characteristics: The task is based on the *Pirate Pain dataset*, where each recording corresponds to one Pirate and provides a multivariate sequence of sensor values collected over time. The dataset mixes continuous features (joint angles, pain survey scores) with categorical or descriptive attributes (e.g., *'two'* eyes or *'one+eye_patch'*). Each sample consists of 160 time steps. The objective is to predict the Pirate's true pain level among three classes: *no_pain*, *low_pain*, and *high_pain*.

Main challenges: One difficulty lies in the variety of input features, as the dataset mixes numerical values with textual attributes. Several variables, such as the number of eyes, hands, or legs, are provided as words rather than numbers, so they must be converted before use. The data is also not normalized, creating inconsistent feature ranges that may affect training stability. Finally, the lack of an official validation split increases the risk of poor model selection or overfitting.

Initial assumptions: Before designing the model, we made a few initial assumptions. We expected proper feature normalization to be necessary for stable training, that preserving class proportions in the train–validation split would be important for reliable evaluation, and that finding a good hyperparameter combination would be challenging given the large number of possibilities.

3 Method

Our methodology can be divided into two main stages: building a functional pipeline and extensive model experimentation.

3.1 Building a Functional Pipeline

The first goal was to create a working codebase that would allow us to train and evaluate models efficiently. We implemented a preprocessing pipeline that included:

Data exploration: We inspected the dataset, checking numerical feature ranges, variable evolution, and identifying non-numerical values.

Mapping categorical values: Pain level labels (*no_pain, low_pain, high_pain*) were converted to 0, 1, and 2. The textual features for eyes, hands, and legs were mapped to numerical values and normalized between 0 and 1.

Train-validation split: The dataset was divided into training and validation sets randomly.

Normalization: All numerical features were normalized to improve training stability and prevent large-magnitude features from dominating the learning process.

3.2 Model Experimentation:

With a working codebase in place, the main focus of our work was manual experimentation. Our goal was to identify effective architectures and hyperparameter configurations.

Model architectures: We tested several recurrent neural network architectures, including RNN, Bidirectional RNN (BiRNN), GRU, BiGRU, LSTM, and BiLSTM models.

Hyperparameter exploration: We experimented with multiple hyperparameters to optimize performance, including the number of layers (L) and neurons per layer (H), learning rate (η), batch size (B), dropout rate (p_{drop}), L1 and L2 regularization weights (λ_1, λ_2), and early stopping patience (P). Different training and validation set sizes were also tested to evaluate the effect of data quantity.
For this part of our experimentations, we also tried to explore these hyperparameters using a grid-search. However, this approach did not yield better results than those obtained manually, while being extremely time-consuming and quickly exhausting the free GPU quota available on Google Colab.

Cross-validation: To reduce bias caused by a single train-validation split, we applied k-fold cross-validation. This ensured that each sample contributed to both training and validation, providing a more robust estimate of model performance.

4 Experiments

Once the experiments were completed on all six model architectures introduced in 3.2, we gathered their accuracy and F1-score in Table 1, in order to determine the best one.

Based on the results obtained, we can see that the LSTM and Bi-GRU models were the most accurate with respect to the task to complete.
The tensorboard for all models has also been calculated and is shown in Figure 1.

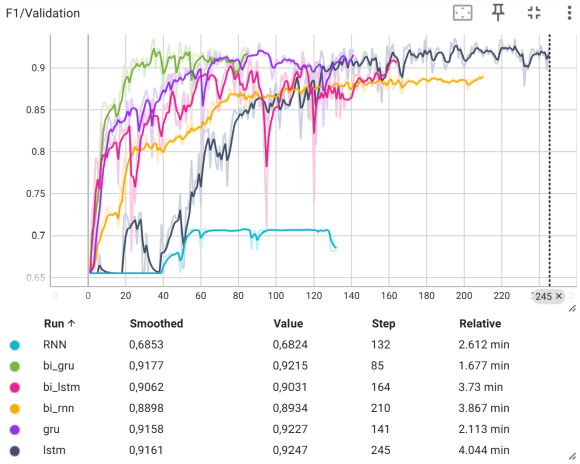

Figure 1: Tensorboard of the six experimented models

5 Results

Our best model achieved an F1-score of **92.95%** on the test set, outperforming the baseline. It was a GRU with a window size of 100, a stride of 20, a learning rate of $1e-3$, and 5 layers of 128 neurons each.

Despite this good result, we observed a few unexpected behaviours. In particular, some recurrent architectures (RNN) showed no meaningful learning progress, with training curves remaining flat or unstable. This suggests that these models were either unsuited to the data structure or insufficiently expressive for the task.

Table 1: F1-score and accuracy of the different models for the validation set

Model	LSTM	Bi-LSTM	RNN	Bi-RNN	GRU	Bi-GRU
F1-score	**0.9358**	0.9200	0.7085	0.8942	0.9304	**0.9356**
Accuracy	**0.9357**	0.9241	0.7731	0.8975	0.9337	**0.9378**

6 Discussion

1. Strengths: A strength of our approach is the breadth of manual experimentation we carried out. We tested six recurrent architectures (GRU, BiGRU, LSTM, BiLSTM, RNN, and BiRNN), and explored many different configurations for each of them. Throughout these trials, we adjusted multiple aspects of the models and the training procedure in an effort to identify promising patterns. Although the results did not always meet our expectations, this exploratory phase allowed us to better understand the limitations of the different architectures on this dataset.

2. Weaknesses:

Hyperparameter search: While we explored many configurations manually, a more systematic grid search would likely have produced better results. Testing all combinations of batch size, dropout rate, L1 and L2 regularisation strengths, window size, stride, train/validation split ratios, learning rate, the value of K for the shuffle procedure, number of layers, number of neurons per layer, and model type (GRU, BiGRU, LSTM, BiLSTM, RNN, BiRNN) would have provided the optimal configuration. Such a search would have been computationally very expensive but could have led to a more performant final model.

Class imbalance: The split between training and validation sets was not perfectly balanced across classes. Although we attempted to preserve the proportions, they still differ slightly. A summary of these proportions is shown below in Table 2:

Table 2: Class distribution in training and validation sets.

Class	**Train**	**Validation**
no_pain	77.65 %	75.94 %
low_pain	14.02 %	15.04 %
high_pain	8.33 %	9.02 %

Exploitation of k-shuffle results: Although we implemented a k-shuffle procedure to reduce bias from a single train/validation split, we did not fully exploit its results. After performing the k shuffles, we could have retrained a final model on the full training set using the best hyperparameters identified. Instead, we only validated on the shuffled splits without performing this final retraining, limiting the potential performance on the test set.

7 Conclusions

In this project, we focused on predicting the pain levels of subjects in the *Pirate Pain dataset* using deep learning models trained entirely from scratch. Our main contributions include the development of a data processing pipeline that handled both numerical and textual features, the implementation of multiple recurrent architectures (RNN, BiRNN, GRU, BiGRU, LSTM, BiLSTM), and manual experimentation with different hyperparameter configurations. These efforts allowed us to achieve an F1-score of 92.95% on the test set; however, this performance was not sufficient to secure a good position on the leaderboard.

Improvements and Future Work: Future improvements could address the weaknesses identified in our study. In particular, a systematic grid search over all relevant hyperparameters would likely lead to a more optimal model configuration. Additionally, ensuring perfectly balanced class proportions between the training and validation sets could reduce bias and improve the reliability of performance estimates. By implementing these strategies, it would be possible to further enhance the model's predictive performance and obtain more robust results on unseen data.